

DEVELOPER GUIDE

VulnScan Toolkit

Setup, Project Structure & Contribution Guide

Version 0.1 (MVP) · July 2026
Author: Higor Silva

1. Prerequisites

- Docker & Docker Compose (recommended way to run and develop against the tool)
- Python 3.11+ (only needed for local, non-containerized development)
- Git

Always develop and test against a lab/authorized target (e.g. DVWA, OWASP Juice Shop, WebGoat run locally). Never point the tool at a target you are not explicitly authorized to test — `config/scope.yaml` will block it anyway.

2. Getting Started

```
git clone <repo-url> vulnscan-toolkit
cd vulnscan-toolkit
docker compose build
docker compose run --rm vulnscan --help
```

For local (non-Docker) development, install dependencies directly and make sure `sqlmap`, `Nikto`, `Nuclei` and the `ZAP` baseline script are available on your `PATH` :

```
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
cd src && python main.py --help
```

3. Project Structure

```
vulnscan-toolkit/
├─ Dockerfile                # Container image: installs Python + all external tools
├─ docker-compose.yml        # Build/run wrapper, mounts ./reports and ./config
├─ requirements.txt
├─ config/
│   └─ scope.yaml            # Target allowlist (edit this before scanning anything)
│   └─ settings.yaml         # Timeouts and tunable parameters
├─ reports/                  # Scan output, one subfolder per run_id
└─ src/
    ├─ main.py               # CLI entry point
    ├─ core/
    │   └─ finding.py         # Finding + Severity model (the module <-> report
contract)
    │   └─ scope_guard.py     # Allowlist enforcement
    │   └─ orchestrator.py   # Resolves and runs modules
    └─ modules/
        └─ base.py            # BaseScanModule abstract class
```

```
|   ├── registry.py          # MODULE_REGISTRY + PROFILES
|   ├── recon.py, sqli.py, xss.py, xxe.py
└── report/
    ├── generator.py        # Aggregation + rendering
    └── templates/report.html.j2
```

4. Core Concepts

4.1 The Finding contract

Every scan module must return a `list[Finding]` (defined in `core/finding.py`). This is the only interface between a module and the report engine — never bypass it by writing directly to the report or by returning raw tool output.

```
Finding(
    module="xss", title="...", description="...",
    severity=Severity.HIGH, cvss_vector="...", cvss_score=7.1,
    endpoint="https://target/app", parameter="q",
    evidence="...", recommendation="...",
    references=[...], tool_source="owasp-zap",
)
```

4.2 BaseScanModule

Every module inherits from `modules/base.py::BaseScanModule` and implements a single method: `run(self) -> list[Finding]`. The base class provides `self._run_cli(cmd, timeout)`, a helper that runs an external CLI command via `subprocess` and persists its stdout/stderr into `self.work_dir` for later inspection.

5. Adding a New Scan Module

To integrate a new tool or vulnerability class (e.g. a CSRF checker, or an SSRF module using a different tool):

1. Create `src/modules/<name>.py` with a class inheriting from `BaseScanModule`.
2. Implement `run()`: build the CLI command, call `self._run_cli(...)`, parse the tool's output (JSON/JSONL/regex on stdout — whatever the tool provides), and return a list of `Finding` objects.
3. Register the class in `src/modules/registry.py` under `MODULE_REGISTRY`, and add it to the relevant entries in `PROFILES` if it should run by default under `quick / full`.
4. If the tool needs to be installed in the container, add the install step to the `Dockerfile`.
5. Add a unit test for the parser under `tests/`, feeding it a saved sample of the tool's raw output.

No changes to `orchestrator.py` or `report/generator.py` should ever be required to add a module — if you find yourself editing those files just to plug in a new tool, the module isn't following the contract correctly.

6. Coding Conventions

- Type hints on all public functions/methods; use `from __future__ import annotations` for forward references.
- Modules should never raise uncaught exceptions that would stop the whole run — the orchestrator already wraps each module call in a try/except and logs failures, but modules should handle expected tool failure modes (timeouts, missing output files) gracefully and return an empty list rather than propagate.
- Never write directly to `self.work_dir`'s stdout/stderr files except through `_run_cli` — keep raw tool output centralized for auditability.
- Keep parsing logic (regex, JSON schema assumptions) isolated in the module file, not leaked into `core/`.

7. Testing

Unit tests live under `tests/` and use `pytest`. Prioritize testing the parsing logic of each module against saved sample outputs (fixtures) rather than running the actual external tools in CI, since the latter requires network access to a real target and is slow/non-deterministic.

```
pip install pytest
pytest tests/ -v
```

For end-to-end validation, run the full pipeline against a local vulnerable app (DVWA, Juice Shop, WebGoat) spun up in a separate container/network, and inspect the generated report for correctness.

8. Extending the Report Template

The report template (`report/templates/report.html.j2`) is plain Jinja2 + inline CSS, rendered identically for both the HTML and PDF outputs (PDF via WeasyPrint on the same HTML string). To add a new section, extend the template and, if needed, add supporting aggregate fields in `ReportGenerator._build_summary()`. Keep CSS print-safe (avoid unsupported CSS3 features — WeasyPrint has partial CSS support) and test PDF output specifically, since HTML rendering in a browser can hide layout issues that WeasyPrint will expose.

9. CLI Reference

Flag	Description
<code>--target</code>	Base target URL (required)
<code>--profile</code>	<code>quick</code> / <code>full</code> / <code>custom</code>
<code>--modules</code>	Explicit module list, overrides profile
<code>--out</code>	Output directory for reports (default <code>./reports</code>)
<code>--format</code>	<code>html</code> / <code>pdf</code> / <code>both</code>
<code>--scope-file</code>	Path to the allowlist file
<code>--i-have-authorization</code>	Required confirmation flag; the tool refuses to run without it